

Project 1 on Machine Learning: regression analysis, resampling methods and gradient descent

Tim Herrmann Holmen

FYS-STK4155

Department of Physics

Dette prosjektet er en vandring gjennom metodene innen dataanalyse som har ledet til det vi idag kaller maskinl ring. Hovedm let er   studere i detalj diverse regresjonsmetoder, som ordinary least squares, ridge regresjon og lasso regresjon. Resampling teknikker som bootstrap og kryssvalidering lar oss vurdere og velge den beste metoden for problemet. For   optimalisere parametrene (hvilke parametre er det her snakk om?: det er selve parametrene og hyperparametrene) vil vi bruke gradient descent, og teste ulike metoder for   utf re denne beregningen, som best r av derivasjon. Vi har studert newtons metode og automatiserte derivasjons metoder, for   optimalisere parametrene i modellene. Polynomene vi utvikler skal passe til en spesifikk en dimensjonal funksjon kalt Runges funksjon, med et st y element, dermed har vi det analytiske svaret tilgjengelig.

CONTENTS

I. Introduction	1
A. 2a The motivation.	1
B. 2b What has been done and what i do.	2
1. Linear Models	2
2. Resampling techniques	2
3. Optimization	2
4. Cross-Validation	3
C. The structure of the report,	3
II. Methods	3
A. The data	3
1. Data processing: Splitting and scaling	3
B. Linear Regression	3
1. Ordinary Least Squares	3
2. Pseudocode OLS	4
C. Ridge Regression	4
1. Pseudocode Ridge	4
D. Lasso Regression	4
1. Pseudocode Lasso	5
E. Evaluation	5
1. Measures of quality	5
2. Bias-Variance tradeoff	5
3. Bootstrap	5
4. Kfold Cross Validation	6
F. Gradient descent	6
1. Plain gradient descent	6
2. Momentum	7
3. Adagrad	7
4. RMSprop	7
5. Adam	7
6. learning rate	7
G. Lasso gradient descent	7
H. Final	7
References	7

I. INTRODUCTION

A. 2a The motivation.

The reason we want to fit a function to data is that we want to predict the value of the function to new unknown data points. For fitting a function to data points,

we use polynomials, and we want to find the best polynomial that fits the data. The best polynomial is the one that minimizes the mean squared error (MSE) between the predicted values and the true values (for the known data). The MSE can be decomposed into three components: bias, variance and irreducible error.

$$\text{MSE} = \text{Bias}^2 + \text{Var}[\hat{y}] + \text{irreducible noise}$$

The bias measures how far the average prediction is from the truth. The variance measures how much the prediction moves when the training set is redrawn. The irreducible error is the variance of the noise itself, which is baked into the data and the model cannot predict it. (Hjorth-Jensen, 2016, p. 62) The bias-variance tradeoff is the balance between bias and variance, in which we minimize the MSE. We do not want an overfitted model, which has low bias and high variance, nor do we want an underfitted model, which has high bias and low variance. Optimally, we want a model that is just right, with low bias and low variance. When fitting a polynomial to data, we want to find the right degree of the polynomial and the right parameters for each degree. For example if there is a linear relationship between the data points, and we can assume this by looking at the data, we use a second order polynomial to describe the data, and the parameters are the coefficients of the polynomial, which can be changed in accordance with the relationship between the data points. The parameters are found by minimizing the MSE between the model prediction and datapoints, for avoiding overfitting, we can split the data into training and testing sets, to see how well the model performs on unseen data. We can use regularization methods, such as ridge regression and lasso regression, to regularize the model and avoid overfitting, and thus lower the variance, while maintaining a good bias. We can also use resampling methods, such as bootstrap and k-fold cross validation, to estimate the performance of the model on unseen data, and thus choose the best model for the data.

With lasso regression, we do not have the analytical solution for the gradient, as we do for OLS and Ridge, and we have to use gradient descent to find the optimal parameters. “Gradient descent is an optimization algorithm inspired by newtons method, which is used to find the minimum of a function. The function we want to minimize is the cost function, which is the MSE between the predicted values and the true values. The gradient descent algorithm uses the derivative of the cost function to find the direction of steepest descent, and then takes a step in that direction, with a step size determined by the learning rate. We can use different optimization algorithms, such as momentum, adagrad, rmsprop and adam, where the learning rate of the descent is dependent on the iteration number to improve the convergence of gradient descent.” — AI After exploring the different methods, we can compare them with cross validation, and choose the best model for the data.

B. 2b What has been done and what i do.

The linear models with OLS nad Ridge have been implemented, and both have close form solutions, thus they can be solved without gradient descent. Lasso regression has been implemented, but it does not have a close form solution, thus we have to use gradient descent to find the optimal parameters. The project starts with the implementation of OLS and Ridge regression, OLS uses the least square distance between model and data to find the optimal parameters, this is the cost function, and the optimal parameters are found by minimizing the cost function. Ridge regression does the same, but adds a regularization term to the cost function, which penalizes large parameters, and thus avoids overfitting. What ridge removes in variance, it adds in increased bias, which will be discussed as the bias-variance tradeoff. For exploring this theme we have emplyed resampling methods, the bootstrap and K-fold cross validation, to estimate the performance of the model on unseen data, and thus choose the best model for the data. The linear models OLS and Ridge are first treated separately, and then compared to each other with resampling methods to find the best degree of the polynomial, and the best regularization parameter, lambda, for Ridge. For lasso regression, we have to use gradient descent to find the optimal parameters, since it does not have a close form solution. With this end in mind, we have implemented plain gradient descent, inspired by newtons iteration method for solving differentials, and explored different optimization algorithms, such as momentum, adagrad, rmsprop and adam, where the learning rate of the descent is dependent on the iteration number. These methods are first implemented on OLS and Ridge, and compared to the closed form solutions of both, to see how well they perform, in terms of convergence (towards zero difference between

closed form and gradient descent solutions), and data cost (number of iterations). In terms of hyperparameters, Ridge and Lasso both have a regularization term. The degree is a hyperparameter for all three models, and the learning rate is a hyperparameter for gradient descent. This project can be split in two parts, where first the Ridge and ordinary least square regression methods are used and evaluated. Then gradient descent is developed and used on all three methods, including Lasso, and the three methods are evaluated by k-fold cross validation. Underway the analytical solutions and sci-kit learn library are used and compared, to ensure numerical digit error, and in the last analysis only sci-kit learn library is used **RIGHT?**.

1. Linear Models

a. OLS

b. Ridge

c. Lasso

2. Resampling techniques

a. Bootstrap

b. Kfold cross validation

3. Optimization

a. Gradient descent

b. Momentum

c. Adagrad

d. RMSprop

e. Adam

f. Stochastic gradient descent

4. Cross-Validation

C. The structure of the report,

The last paragraph, one sentence per section, and where code can be found. Code can be found on the following Github repository: (Holmen, 2026)

II. METHODS

A. The data

In this project, the input data is normally distributed, and the output data is generated by Runge's function, with added noise, Runge Function which is a one-dimensional function of second order.

$$f(x) = \frac{1}{1 + 25x^2}$$

From the beginning, we have created a relation between the input and output data. This is helpful since the project is research in machine learning methods, it is usefull to know the exact solution. The noise is normally distributed, with a mean of 0 and a variable standard deviation. From the form of the runge function, we can see that the function has a maximum at $x=0$, and the function is symmetric around this point, for values larger than $[-1,1]$, the function appoahes zero. Thus we will focus on data which is normally distributed between -1 and 1. The output is generated by adding noise σ to the runge fucntion defined above, and the noise is normally distributed with a mean of 0 and a standard deviation of σ .

The input value x is transformed into a design matrix X , where the columns describe a polynomial of variable degree d , in such a way that the first column is x^0 , the intrcept, the second column is x^1 , and so on, up to x^d . The rows of the matrix are the data points, and are thus copied and multiplied according to the polynomial degree, or the column of the matrice.

$$\mathbf{X} = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{pmatrix} \in \mathbb{R}^{n \times (d+1)}. \quad (1)$$

1. Data processing: Splitting and scaling

- Train test split of skicit
- Unless explicitly told otherwise, such as in the bootstrap resampling method, the training and test input data are normalized, using training values.
- All random data are within the random seed 2026

a. *Pseudocode Data* Here is a pseudocode for the splitting and scaling of the data:

Listing 1 Preparation of training and test data.

```
FUNCTION PrepareData(x, y, degree):
  X <- DesignMatrix(x, degree)

  Split X and y into:
    70% training data
    30% test data
  Use random seed 2026

  mu <- column means of X_train
  s <- column standard deviations of X_train

  mu[0] <- 0
  s[0] <- 1

  X_train <- (X_train - mu) / s
  X_test <- (X_test - mu) / s

  RETURN X_train, X_test,
         y_train, y_test
```

B. Linear Regression

The basic idea of linear regression is, by using a polynomial $f(x)$ of fixed degree to describe the data points given the input data x , to adjust the coefficients in such a way that the difference between the predicted values of the model $f(x)$ and the true output value $y(x)$. The coefficients are adjusted so that mean of the sum of the difference squared is minimised, which is the basis for the name least squares regression.

1. Ordinary Least Sqaures

Thus the cost function, for least squared, is written mathematically:

$$C(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 \quad (2)$$

(Hjorth-Jensen, 2016, p. 81) This is the optimization problem, to adjust the parameter θ such that the cost function is minimized.

We se that our predicted model is characterized by $X\theta$, where theta contains the coefficients, the direct solution to this problem is:

$$\hat{\theta} = (X^T X)^{-1} X^T y \quad (3)$$

(Hjorth-Jensen, 2016, p. 81) through singular value decomposition (Hjorth-Jensen, 2016, p. 42), it can be shown that the best estimate for this theta, which minimizez the cost function, is the pseudoinverse of X multiplied by the output data y ,

$$\hat{\theta} = X^+ y \quad (4)$$

(Hjorth-Jensen, 2016, p. 82) The pseudoinverse is solved by the numpy package `np.linalg.pinv(X)`, on the assumption that the columns of X are linearly independent, such that $X^T X = I$ and $X^+ = (X^T X)^{-1} X^T = I^{-1} X^T = X^T$.

It is safe to assume that the columns are linearly independent, since the number of columns (degrees d) is much smaller than the number of rows (data points n), the possibility of one column being a linear combination of the other columns is close to zero.

The data is split into training and test data as described above, and the training data is normalised as explained above. This ensures that the model can predict on the test data of X , and the predictions can be used against the test data of y to calculate the mean squared error MSE and coefficient of determination R^2 , how these values are calculated will be explained in [measures of quality](#). This procedure is done for different degrees of the polynomial design matrix. The parameters are plotted against degree, and the predicted models for different degrees are plotted against the runge function, for different values of the noise σ .

2. Pseudocode OLS

Here we go

Listing 2 Ordinary least squares regression.

```
FOR degree = 0, ..., 15:
    X_train, X_test, y_train, y_test
        <- PrepareData(x, y, degree)

    theta <- pseudoinverse(X_train) * y_train

    prediction <- X_test * theta

    mse <- MSE(y_test, prediction)
    r2 <- R2(y_test, prediction)

    Store degree, theta, mse and r2
```

C. Ridge Regression

Ridge regression adds a penalty on the squared length of the parameter vector, and the cost function is;

$$C_{\text{Ridge}}(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2, \quad \lambda \geq 0. \quad (5)$$

with $\|\theta\|_2^2 = \sum_j \theta_j^2$. The closed form solution to this problem is, by dropping the factor $1/n$:

$$\hat{\theta}_{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T y \quad (6)$$

I is the $p \times p$ identity matrix. The penalty adds λ to the eigenvalues of $X^T X$. (Hjorth-Jensen, 2016, p. 87)

Supposing the design matrix is orthonormal, $X^T X = I$: then $\hat{\theta}_{\text{OLS}} = X^T y$ and

$$\hat{\theta}_{\text{Ridge}} = (I + \lambda I)^{-1} X^T y = \frac{1}{1 + \lambda} \hat{\theta}_{\text{OLS}} \quad (7)$$

So every coefficient is scaled by the same factor, and the estimator tends to zero as $\lambda \rightarrow \infty$. Ridge shrinks all coefficients proportionally; it never sets any of the exactly to zero.

1. Pseudocode Ridge

Here we go:

Listing 3 Ridge regression without penalizing the intercept.

```
lambdas <- logspace(-3, 2, 5)

FOR degree = 0, ..., 15:
    X_train, X_test, y_train, y_test
        <- PrepareData(x, y, degree)

    P <- IdentityMatrix(degree + 1)
    P[0, 0] <- 0

    FOR lambda in lambdas:
        A <- transpose(X_train) * X_train
            + lambda * P

        b <- transpose(X_train) * y_train

        theta <- pseudoinverse(A) * b

        prediction <- X_test * theta

        mse <- MSE(y_test, prediction)
        r2 <- R2(y_test, prediction)

        Store degree, lambda, theta, mse and r2
```

D. Lasso Regression

Replacing the 2-norm penalty of Ridge by a 1-norm gives the cost function

$$C_{\text{Lasso}}(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 + \lambda \|\theta\|_1, \quad (8)$$

$$\|\theta\|_1 = \sum_{j=0}^{p-1} |\theta_j|$$

Which is the Lasso - least absolute shrinkage and selection operator. The Lasso sets coefficients exactly to zero, and thereby performs variable selection as part of the fitting. (Hjorth-Jensen, 2016, p. 90) The lasso has no closed form solution, since differentiating the [cost function](#), requires the derivative of the absolute value of θ :

$$\frac{d|\theta|}{d\theta} = \text{sgn}(\theta) = \begin{cases} +1, & \theta > 0, \\ -1, & \theta < 0. \end{cases} \quad (9)$$

This equation is undefined at the origin, $\theta = 0$, thus the closed form solution is lost, but a solution can still be found through the soft thresholding operator and gradient descent methods, which will be explained in [section gradient descent](#). Here is the soft thresholding operator for $X = I$

$$\begin{aligned}\hat{\theta}_i^{\text{Lasso}} &= S_{\lambda/2}(y) \\ &= \text{sgn}(y) \max(|y| - \lambda/2, 0)\end{aligned}\quad (10)$$

(Hjorth-Jensen, 2016, p. 91) The selection property makes the lasso the natural first tool when the number of parameters is large and most features are believed irrelevant. Pseudocode for lasso will be written after gradient descent is described.

1. Pseudocode Lasso

E. Evaluation

1. Measures of quality

a. *Coefficient of determination* This

$$\begin{aligned}R^2 &= 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \\ \bar{y} &= \frac{1}{n} \sum_{i=1}^n y_i.\end{aligned}\quad (11)$$

b. *Mean squared error*

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (12)$$

c. *Bias* For this project we will look at bias squared, the reason for this is that the squared values are positive, such that the terms will not cancel when we take the mean. the equation is.

$$\begin{aligned}\text{Bias}^2 &= \frac{1}{n} \sum_i (f_i - \mathbb{E}[\hat{y}])^2, \\ \mathbb{E}[\hat{y}] &= \frac{1}{n} \sum_i \hat{y}_i\end{aligned}\quad (13)$$

The bias measures how far the average prediction $\mathbb{E}[\hat{y}]$ is from the truth $f(x)$. (Hjorth-Jensen, 2016, p. 61-62)

In the case of this project, the function $f(x)$ is the runge function. The details on this will be discussed in the [bias variance tradeoff](#), since the data $y = f(x) + \epsilon$ includes noise.

d. *Variance* The variance is closely related to the standard deviation, which is the square root of the variance, and tells how much the data varies within the sample.

$$\text{Var}[\hat{y}] = \frac{1}{n} \sum_i (\hat{y}_i - \mathbb{E}[\hat{y}])^2 \quad (14)$$

(Hjorth-Jensen, 2016, p. 62) The variance measures how much the prediction moves when the training set is redrawn. (Feil?)

2. Bias-Variance tradeoff

Before we perform an analysis of the bias variance tradeoff on the test data, we make a figure similar to Fig. 2.11 of Hastie, Tibshirani and Friedman. (Hastie et al., 2017, p. 38) This figure displays only the test and training MSE. The test MSE can be used to indicate possible regions of low/high bias and variance. To solve this, different values of noise, datapoints and polynomial degree is explored.

The bias variance tradeoff equation:

$$\begin{aligned}\mathbb{E}[(\mathbf{y} - \hat{\mathbf{y}})^2] &= \frac{1}{n} \sum_{i=1}^n (f_i - \mathbb{E}[\hat{y}_i])^2 \\ &\quad + \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - \mathbb{E}[\hat{y}_i])^2 + \sigma^2\end{aligned}\quad (15)$$

$$\text{MSE} = \text{Bias}^2 + \text{Var}[\hat{y}] + \text{irreducible noise} \quad (16)$$

Our empirical bias-related term compares the mean prediction with the noisy test targets, rather than with the noise-free function. Consequently, it also contains a contribution from the test noise and should not be interpreted as the squared bias alone.

a. *Fig 4.17*

3. Bootstrap

We study the bias variance tradeoff by implementing the bootstrap resampling technique. For studying this tradeoff we will only use ordinary least square for simplicity. The bootstrap approximates the sampling distribution of an estimator by repeatedly sampling from the observed data.

1. Draw with replacement n values from the observations $\mathbf{x} = (x_0, \dots, x_{n-1})$, forming a resample $\mathbf{x}^*$
2. Evaluate the estimator on the resample to obtain $\hat{\theta}^*$

3. Repeat k times

(Hjorth-Jensen, 2016, p. 65) The gist is that since $\hat{\theta} = \theta(\hat{\mathbf{X}})$ is a function of random variables, theta itself is a random variable with some probability distribution function $p(t)$. The different bootstrap samples, pulls different random samples from the training data, with replacement, and each produce different fitted coefficients and therefore different predictions. The number of samples is chosen as a variable, in our case 40, 100 and 300 samples, and all the models are used to calculate the man squard error, bias and variance between all the fitted models and the test data. This is done for each degree of the polynomial. To pull the random data from the training data, we use sci-kit function `resample` for pulling the random bootstrap data.

a. *Bootstrap Pseudocode* LLM Declaration: This pseudocode is a pseudo copy of the reference project code c: Bootstrap (Holmen, 2026) created with help of OpenAI codex (OpenAI, nd)

Listing 4 Bootstrap estimation of prediction error, a bias-related term, and variance for OLS.

```
FUNCTION BootstrapBiasVariance(n_bs, maxdeg,
seed):
  Initialize random generator using seed
  Generate n_bs bootstrap seeds

  error      <- zeros(maxdeg + 1)
  bias       <- zeros(maxdeg + 1)
  variance   <- zeros(maxdeg + 1)

  FOR degree = 0, ..., maxdeg:
    X <- DesignMatrix(x, degree)

    Split X and y into:
      70% training data
      30% test data
    Use random seed 2026

    n_test <- number of test observations
    predictions <- zeros(n_test, n_bs)

    FOR b = 0, ..., n_bs - 1:
      Draw paired rows from X_train and
        y_train
      with replacement
      using bootstrap seed b
      and preserving the training set
        size

      X_bs, y_bs <- bootstrap sample

      theta <- pseudoinverse(X_bs) * y_bs

      predictions[:, b] <- X_test * theta

  mean_prediction <- row means of
    predictions
  point_variance  <- row population
    variances
                                of predictions
```

```
targets <- repeat y_test in n_bs columns

error[degree] <- mean(
  (targets - predictions)^2
)

bias[degree] <- mean(
  (y_test - mean_prediction)^2
)

variance[degree] <- mean(point_variance)

RETURN error, bias, variance
```

4. Kfold Cross Validation

The data are divided into k roughly equal, mutually exclusive and exhaustive subsets, the folds. Each in turn plays the role of the test set while the union of the k-1 folds serve as the training set. Each observation is therefore used for testing exactly once and for training exactly k-1 times. The procedure is:

1. Shuffle the data set at random
2. Split into k folds.
3. For each fold: hold it out, fit the model on the remaining folds, evaluate on the held out folds and discard the model.
4. Summarize the model by the mean, the spread, of the k scores.

(Hjorth-Jensen, 2016, p. 66) The k fold cross validation is solved with closed form solutions for ridge and OLS, and with ski-kit functions Linearregression and Ridge. The mean value of the mean squared error of all folds is calculated analytically and with sci-kit function cross_val_score, and the difference between analytical and sci-kit solutions are calculated. 5 fold and 10 fold cross validation is used, and the ridge regression is calculated for different values of lambda, the penalty. The sci-kit method is used to ensure that the results agree with the results obtained numerically.

The cross validation is used to find the best degree, the best lambda value for ridge, and the best overall model, also the difference between the 5 fold and the 10 fold cross validation result is shown.

F. Gradient descent

1. Plain gradient descent

The gradient descent methods introduced is a method for finding the optimal parameter θ , and is thus a replacement for the closed form solutions of OLS and Ridge, and gives a solution for the lasso regression problem, which

had no analytical CF solution. In this regard the previous solutions for OLS and ridge can be used to verify that the gradient descent methods converge to the analytical solutions.

In its simplest form, the gradient descent method can be written

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} C(\theta_k) \quad (17)$$

(Hjorth-Jensen, 2026) The gradients of the cost function, will be solved both analytically and with automatic differentiation (JAX), and the results will be compared to prove that they agree to numerical precision.

Initially the learning rate η is guessed and adjusted for convergence over the number of iterations. But the learning rate has some limits, as long as the Hessian is known, as it is for Ridge and OLS, we can calculate this limit, the limit of convergence: For all directions of the cost function to converge we need $0 < \eta < \frac{2}{\lambda_{max}}$, where λ_{max} is the largest eigen value of the Hessian. Since the Hessian is the second derivative of the cost function, the largest eigenvalue is the steepest direction, and the smallest eigenvalue is the slowest direction, thus we can define the condition number, $\kappa = \frac{\lambda_{max}}{\lambda_{min}}$, which is proportional to the number of iterations needed to reduce the error by a fixed factor. The bound is set by the steepest direction, and a flat direction (0). Presumably the number of iterations reaches its minimum at $\eta = \frac{2}{\lambda_{min} + \lambda_{max}}$, and diverges exactly at $\frac{2}{\lambda_{max}}$.

Momentum, Adagrad, RMSprop and Adam are advanced

gradient descent methods where the learning rate $\eta(n)$ is updated during the iteration. Opposed to plain gradient descent where the learning rate is constant. The more advanced methods are more suitable for cases where we cannot analytically calculate the Hessian of the cost function.

2. Momentum

3. Adagrad

4. RMSprop

5. Adam

6. learning rate

G. Lasso gradient descent

H. Final

REFERENCES

- T. Hastie, R. Tibshirani, and J. Friedman (2017), *The Elements of Statistical Learning* (Springer).
- M. Hjorth-Jensen (2016), *Machine Learning and the Physical Sciences* (Springer).
- M. Hjorth-Jensen (2026), "Project 1 on Machine Learning,".
- T. Holmen (2026), "Prosjekt1," <https://github.com/Timholmen96/FYS-STK4155>.
- OpenAI (n.d.), "Codex,".